
Ensemble Methods

Cap 7 HML

Ensemble Methods

Construct a set of classifiers from the training data

Predict class label of previously unseen records by aggregating predictions made by multiple classifiers

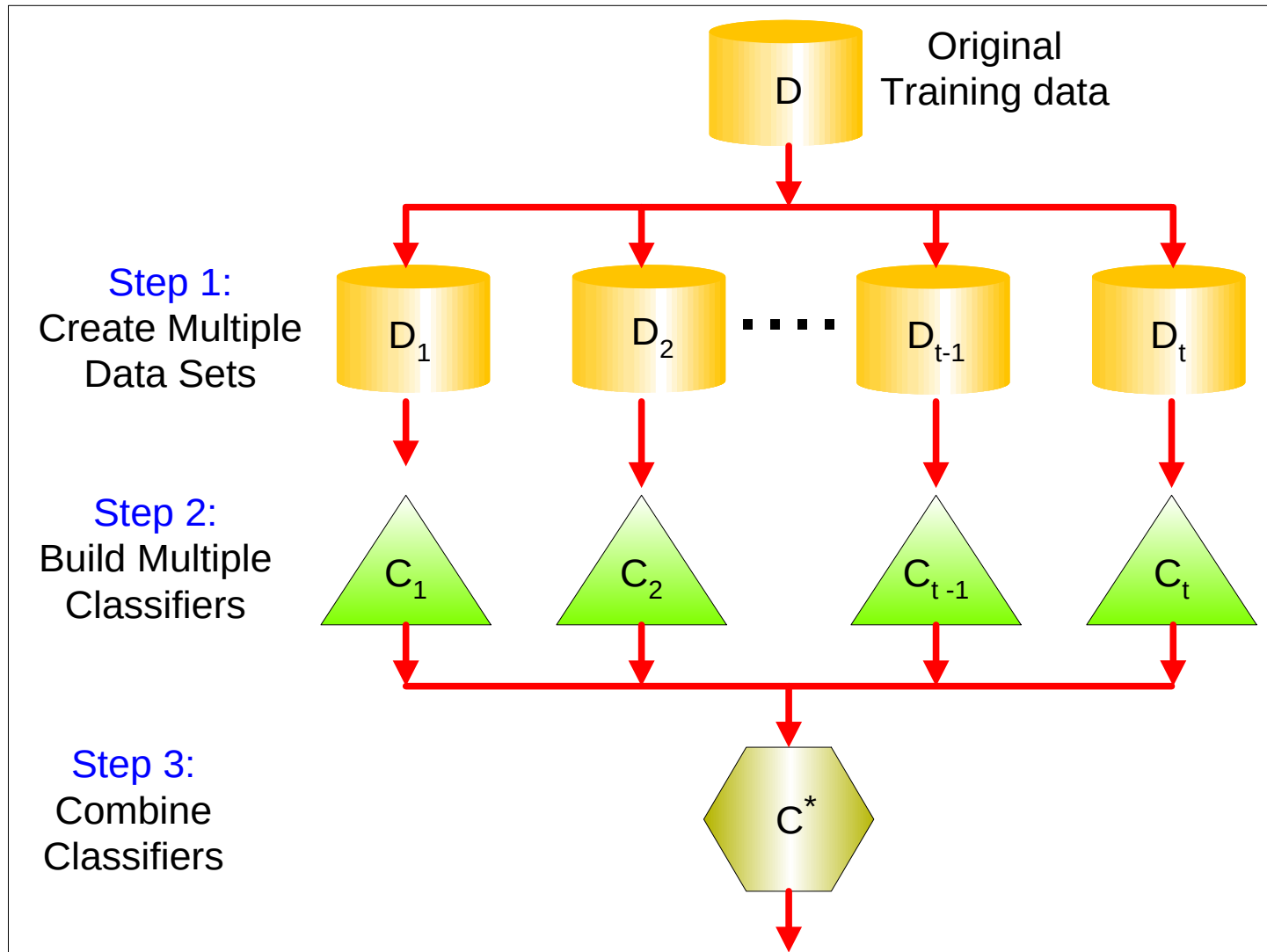
Ensemble learning

- Combining multiple models
 - ▣ The basic idea
- Bagging
 - ▣ Bias-variance decomposition, bagging with costs
- Randomization
 - ▣ Random forests
- Boosting
 - ▣ AdaBoost, the power of boosting
- Stacking

Combining multiple models

- Basic idea:
build different “experts”, let them vote
- Advantage:
 - ❑ often improves predictive performance
- Disadvantage:
 - ❑ usually produces output that is very hard to analyze
 - ❑ but: there are approaches that aim to produce a single comprehensible structure

General Idea



Why does it work?

Suppose there are 25 base classifiers

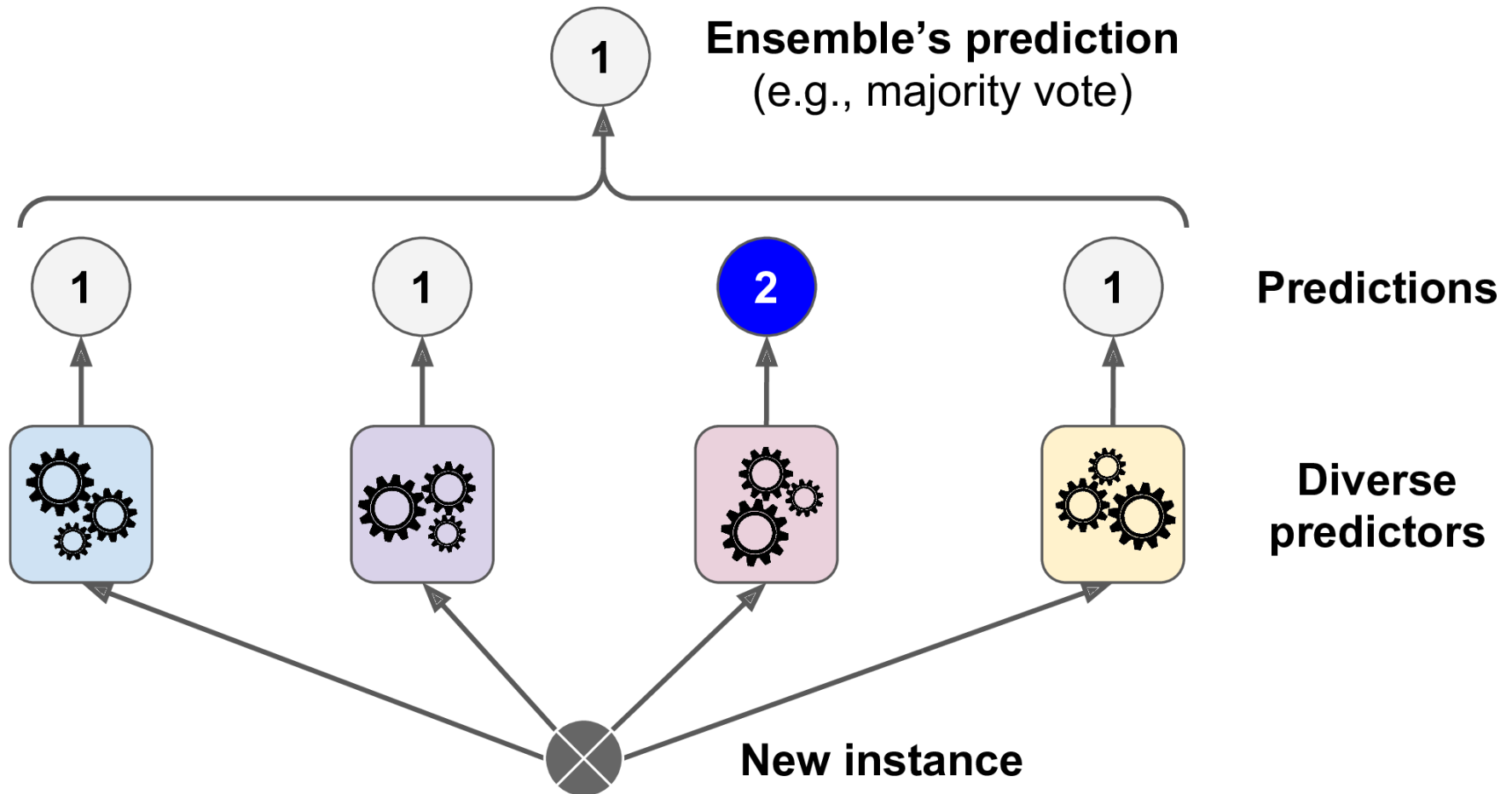
- Each classifier has error rate, $\varepsilon = 0.35$
- Assume classifiers are independent
 - i.e., probability that a classifier makes a mistake does not depend on whether other classifiers made a mistake
 - **Note:** in practice they are not independent!

Probability that the ensemble classifier makes a wrong prediction:

- The ensemble makes a wrong prediction if the majority of the classifiers makes a wrong prediction
- The probability that 13 or more classifiers are wrong is

$$\sum_{i=13}^{25} \binom{25}{i} \varepsilon^i (1-\varepsilon)^{25-i} = 0.06 \ll \varepsilon$$

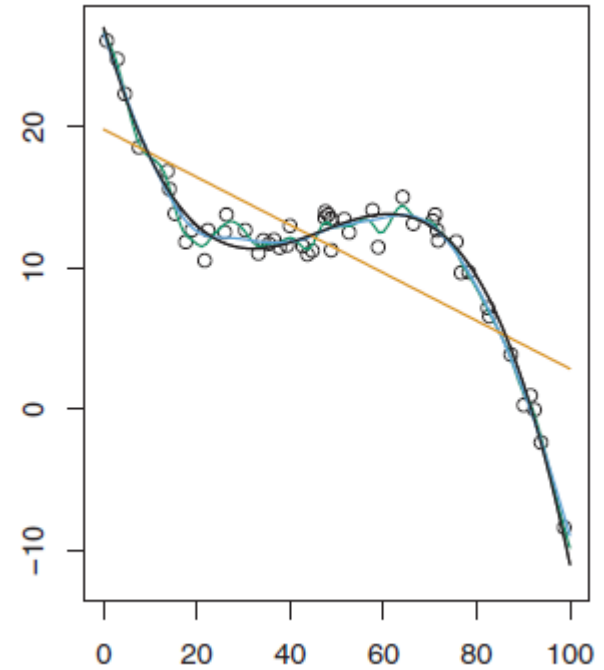
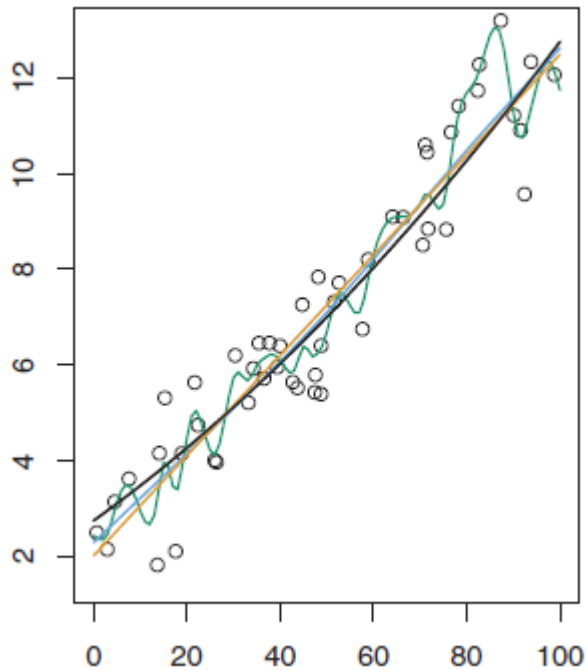
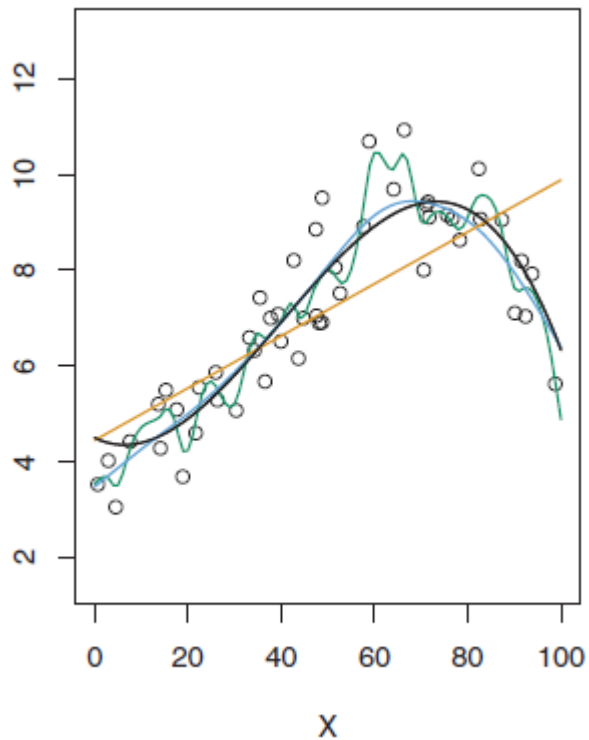
Hard voting classifier predictions



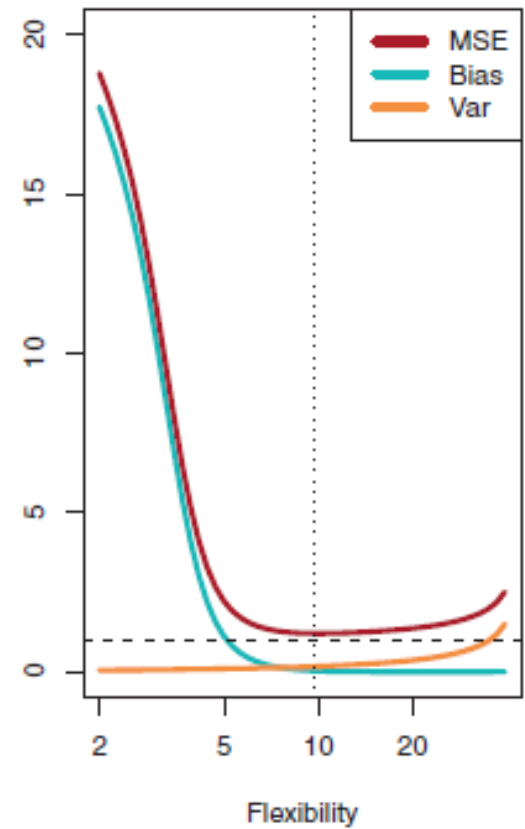
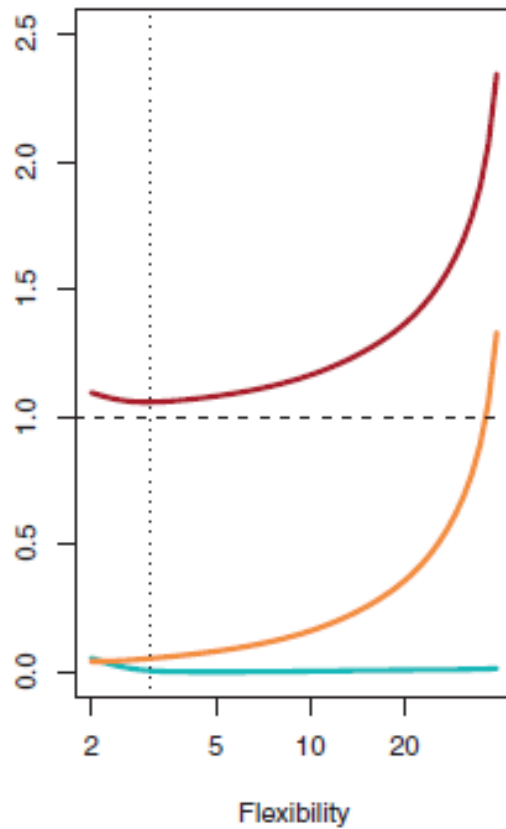
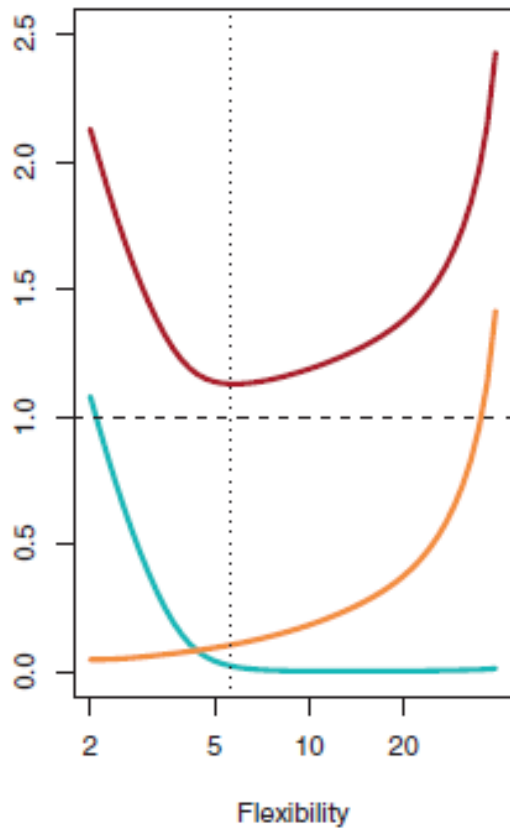
Bias-variance decomposition

- Used to analyze how much selection of any *specific* training set affects performance
- Assume infinitely many classifiers, built from different training sets of size n
- For any learning scheme,
 - *Bias* = expected error that is introduced by approximating a real-life problem, which may be extremely complicated, by a much simpler model
 - *Variance* = expected error due to the particular training set used
- Total expected error = bias + variance + $\text{Var}(\varepsilon)$

Bias-variance decomposition



Bias-variance decomposition



Bagging

Bootstrap Aggregation=Bagging

- Combining predictions by voting/averaging
 - Simplest way
 - Each model receives equal weight
- “Idealized” version:
 - Sample several training sets of size n
(instead of just having one training set of size n)
 - Build a classifier for each training set
 - Combine the classifiers’ predictions
- Learning scheme is *unstable* $\Rightarrow$
almost always improves performance
 - Small change in training data can make big change in model (e.g. decision trees)

More on bagging

- Bagging works because it reduces *variance* by voting/averaging
 - Note: in some pathological hypothetical situations the overall error might increase
 - Usually, the more classifiers the better
- Problem: we only have one dataset!

More on bagging

- Solution: generate new ones of size n by sampling from it *with replacement*
- Can help a lot if data is noisy
- Can also be applied to numeric prediction
 - Aside: bias-variance decomposition originally only known for numeric prediction

Bagging classifiers

Model generation

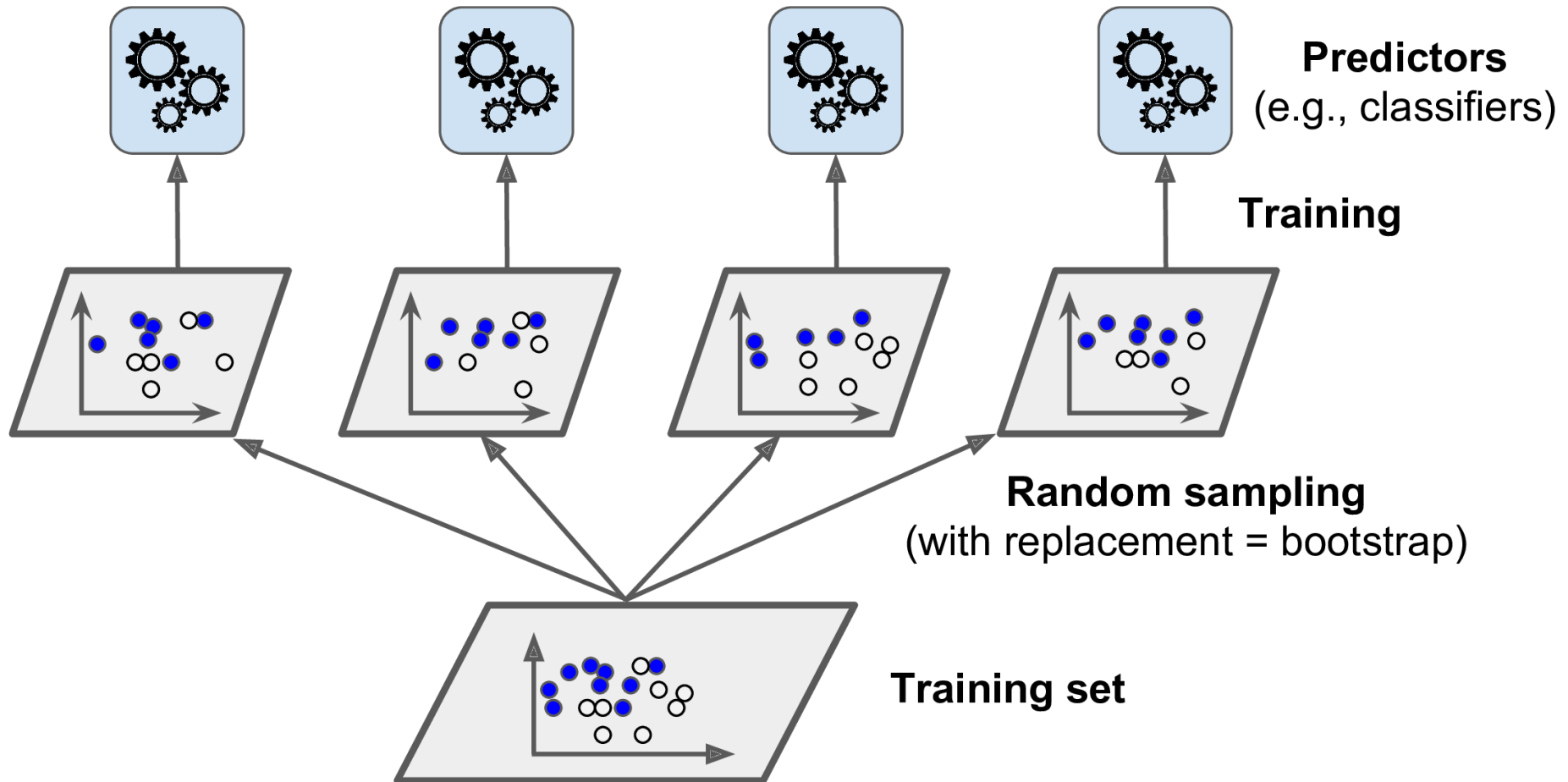
```
Let  $n$  be the number of instances in the training data
For each of  $t$  iterations:
  Sample  $n$  instances from training set
    (with replacement)
  Apply learning algorithm to the sample
  Store resulting model
```

Classification

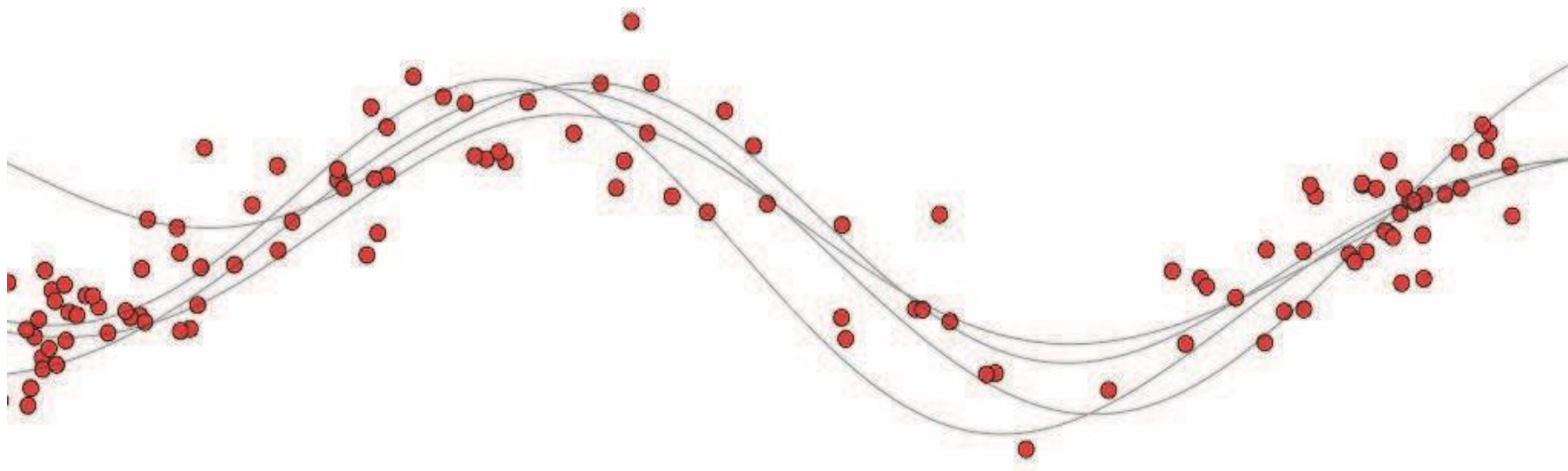
```
For each of the  $t$  models:
  Predict class of instance using model
Return class that is predicted most often
```

When sampling is performed *without* replacement,
it is called *Pasting*

Bagging/Pasting



A concrete example



We select some input samples

We learn a regression model

Very sensitive to the input selection

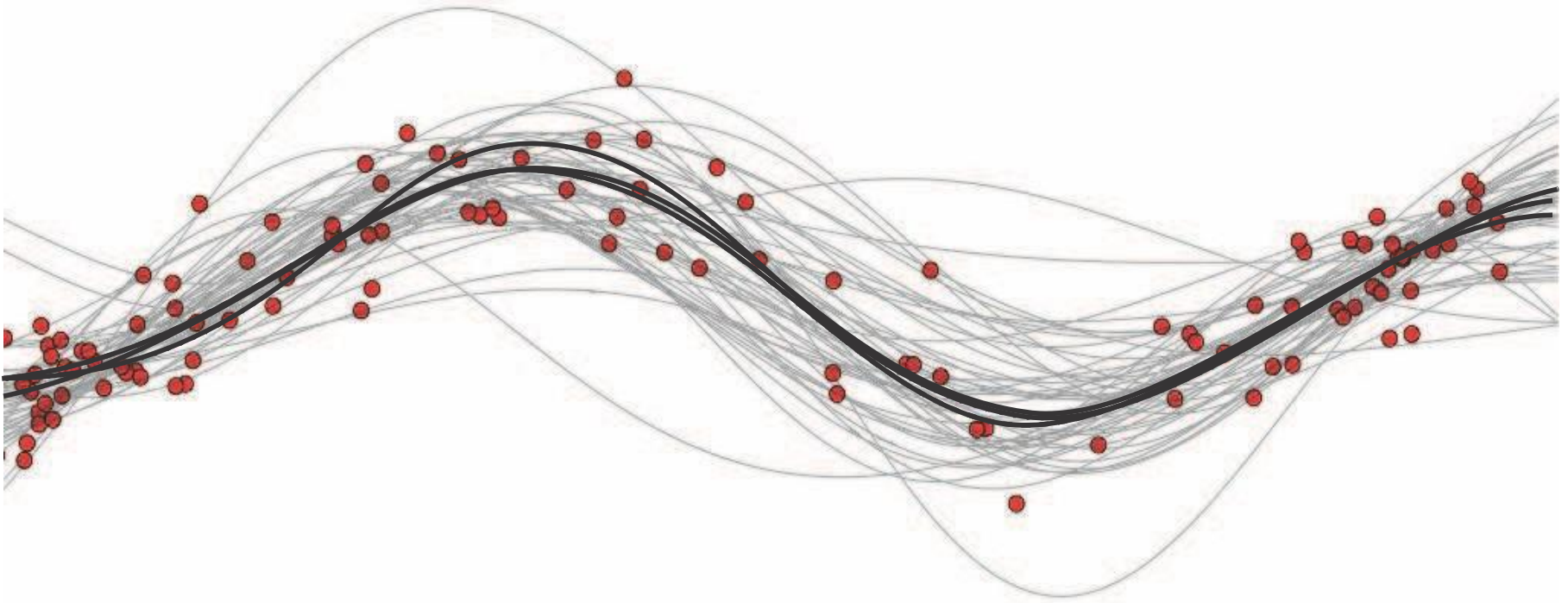
m training sets = m different models

$$(x_1^{(1)}, y_1^{(1)}), \dots, (x_n^{(1)}, y_n^{(1)}) \rightarrow \hat{f}^{(1)}(x) = Y^{(1)}$$

$$(x_1^{(2)}, y_1^{(2)}), \dots, (x_n^{(2)}, y_n^{(2)}) \rightarrow \hat{f}^{(2)}(x) = Y^{(2)}$$

$$\hat{f}^{(1)}, \dots, \hat{f}^{(m)} \rightarrow \begin{matrix} Y^{(1)}, \dots, Y^{(m)} \\ Y \end{matrix}$$

Aggregation: combine several models



- Linear combination of simple models
- More examples = Better model
- We can stop when we're satisfied

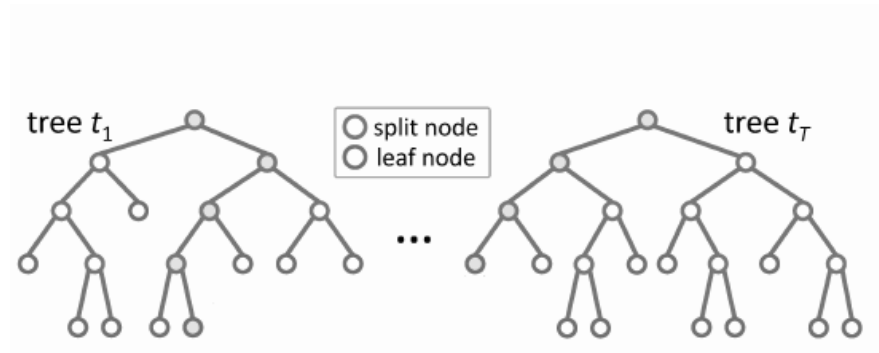
Randomization

- Can randomize learning algorithm instead of input
- Some algorithms already have a random component: eg. initial weights in neural net
- Most algorithms can be randomized, eg. greedy algorithms:
 - Pick from the N best options at random instead of always picking the best options
 - Eg.: attribute selection in decision trees
- More generally applicable than bagging: e.g. random subsets in nearest-neighbor scheme
- Can be combined with bagging

Random Patches and Random Subspaces

- The `BaggingClassifier` class supports sampling the features as well (using the `max_features` and `bootstrap_features` hyperparameters), i.e., each predictor will be trained on a random subset of the input features.
- This is particularly useful when you are dealing with high-dimensional inputs.
- Sampling both training instances and features is called the *Random Patches* method.
- Keeping all training instances but sampling features is called the *Random Subspaces* method

Random Forests



- Random forests (RF) are a combination of tree predictors
- The generalization error depends on the strength of the individual trees and the correlation between them

The Random Forests Algorithm

Given a training set S

For $i = 1$ to k do:

 Build subset S_i by sampling with replacement from S

 Learn tree T_i from S_i

 At each node:

 Choose best split from random subset of F features

 Each tree grows to the largest extend, and no pruning

Make predictions according to majority vote of the set of k trees.

Features of Random Forests

- It can provide good accuracy.
- It runs efficiently on large data bases.
- It can handle thousands of input variables without variable deletion.
- It gives estimates of what variables are important in the classification.
- It has an effective method for estimating missing data and maintains accuracy when a large proportion of the data are missing.

Extra Trees

Extra Trees is like Random Forest, in that it builds multiple trees and splits nodes using random subsets of features, but with two key differences:

- builds multiple trees with **bootstrap = False** by default, which means it samples without replacement
- nodes are split based on **random** splits among a **random subset** of the features selected at every node

In Extra Trees, randomness doesn't come from bootstrapping of data, but rather comes from the random splits of all observations.

ExtraTrees is named for (**Extremely Randomized Trees**).

Out-of-Bag Evaluation

- Since a predictor never sees the **oob** instances during training, it can be evaluated on these instances, without the need for a separate validation set.
- You can *evaluate the ensemble* itself by averaging out the oob evaluations of each predictor.
- In Scikit-Learn, you can set `oob_score=True` when creating a `BaggingClassifier` to request an automatic oob evaluation after training

Feature Importance

When a tree is built, the decision about which variable to split at each node uses a calculation of the Gini impurity.

For each variable, the sum of the Gini decrease across every tree of the forest is accumulated every time that variable is chosen to split a node.

Moreover, each node has a weight that is proportional to the number of training samples that are associated with it.

Scikit-Learn computes this score automatically for each feature after training and stores it in the `feature_importances_` variable.

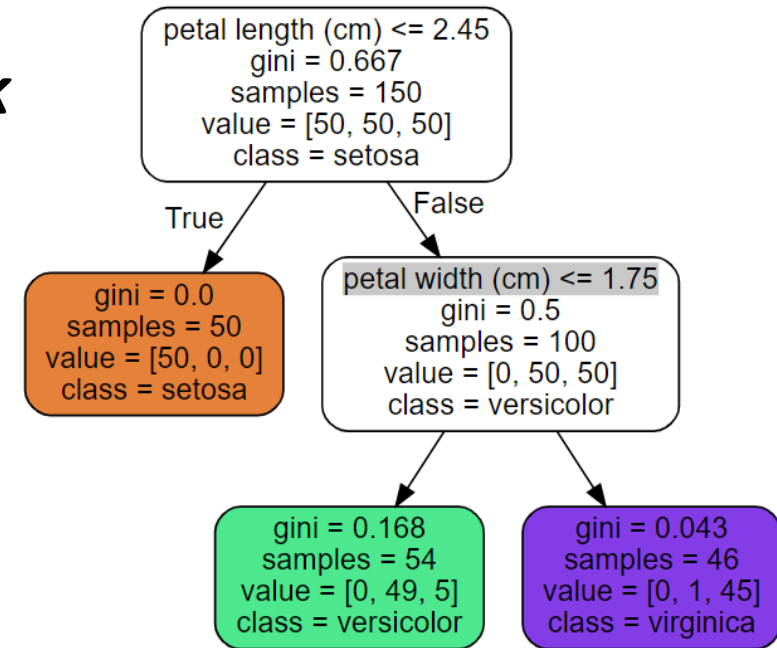
RF Feature Importance

ni_j^k : importance of node j of tree k

w_j : weighted number of samples reaching node j

C_j : the impurity value of node j

$left/right(j)$: child node from left/right split on node j



$$ni_j^k = w_j C_j - w_{left(j)} C_{left(j)} - w_{right(j)} C_{right(j)}$$

$$fi_j^k = \frac{\sum_{i: \text{node } i \text{ splits on features } j} ni_i^k}{\sum_{i \in \text{all nodes}} ni_i^k}$$

RF Feature Importance

fi_j^k values can then be normalized to a value between 0 and 1 by dividing by the sum of all feature importance values:

$$normfi_j^k = \frac{fi_j^k}{\sum_{i \in all\ features} fi_i^k}$$

The final feature importance, at the Random Forest level, is its (normalized) average over all the trees. See this [link](#).

$$RFfi_j = \frac{\sum_{k \in all\ trees} normfi_j^k}{number\ of\ trees}$$



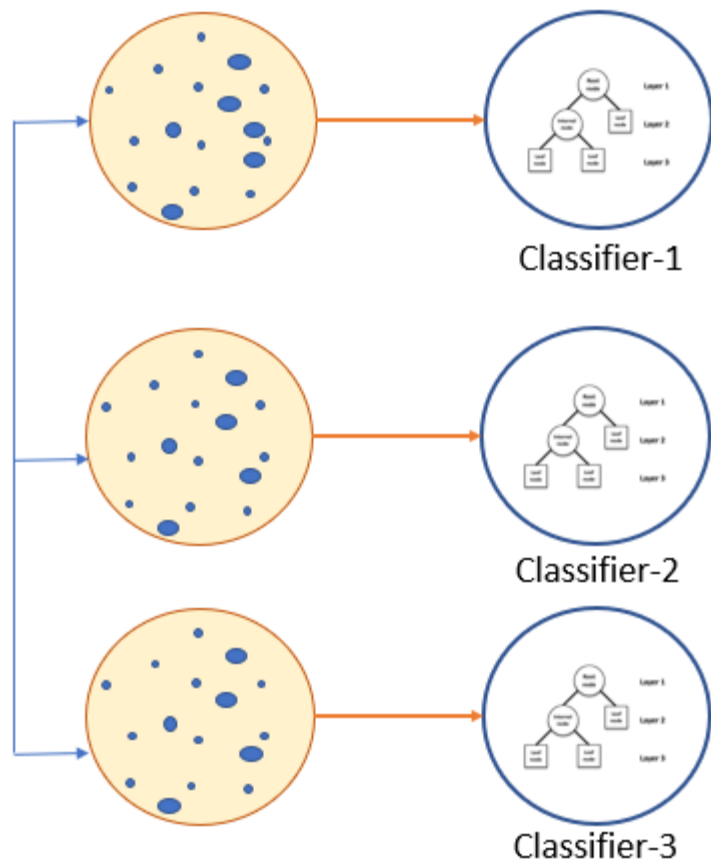
Boosting

Boosting

- Also uses voting/averaging
- Weights models according to performance
- Iterative: new models are influenced by performance of previously built ones
 - ❑ Encourage new model to become an “expert” for instances misclassified by earlier models
 - ❑ Intuitive justification: models should be experts that complement each other
- Several variants

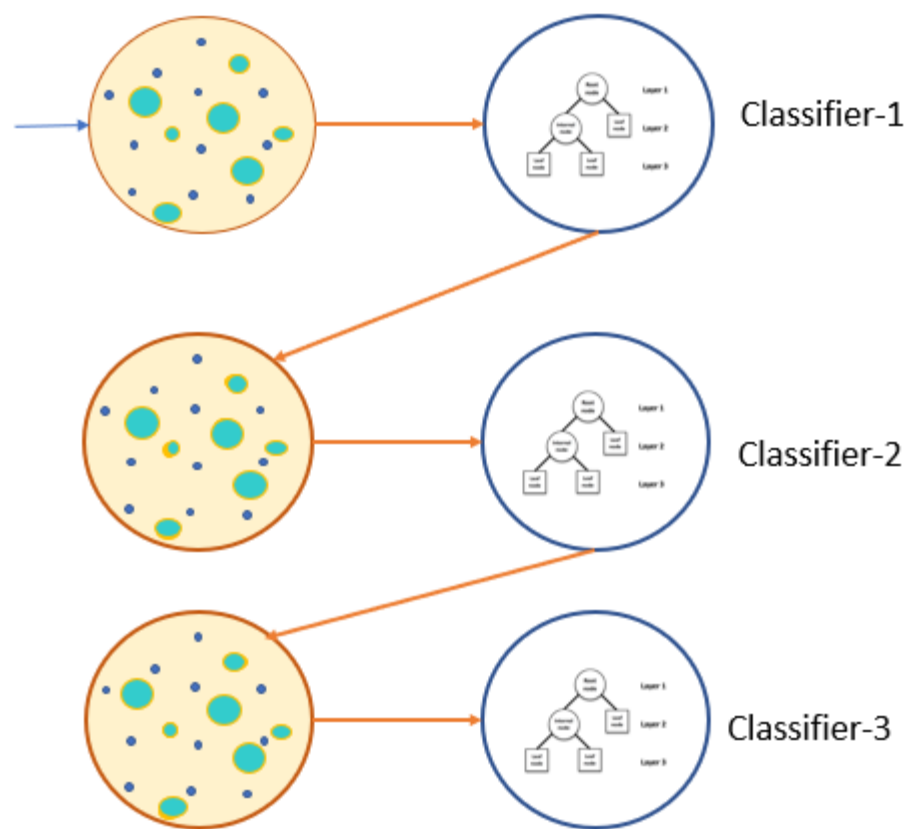
Boosting

Bagging



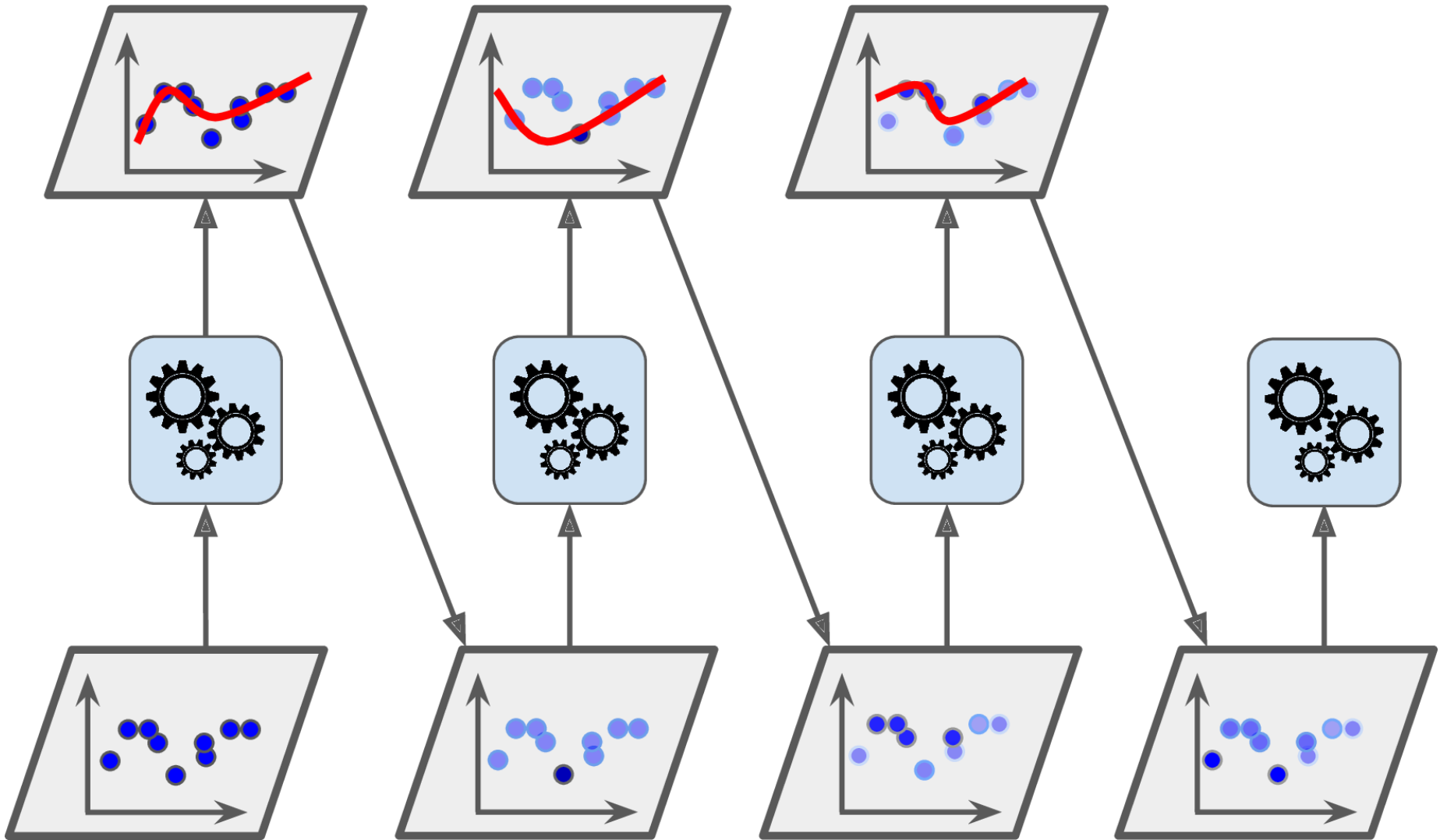
Parallel

Boosting



Sequential

Boosting



Boosting for Regression Trees

Algorithm 8.2 *Boosting for Regression Trees*

1. Set $\hat{f}(x) = 0$ and $r_i = y_i$ for all i in the training set.
2. For $b = 1, 2, \dots, B$, repeat:
 - (a) Fit a tree $\hat{f}^b$ with d splits ($d + 1$ terminal nodes) to the training data (X, r) .
 - (b) Update $\hat{f}$ by adding in a shrunk version of the new tree:

$$\hat{f}(x) \leftarrow \hat{f}(x) + \lambda \hat{f}^b(x). \quad (8.10)$$

- (c) Update the residuals,

$$r_i \leftarrow r_i - \lambda \hat{f}^b(x_i). \quad (8.11)$$

3. Output the boosted model,

$$\hat{f}(x) = \sum_{b=1}^B \lambda \hat{f}^b(x). \quad (8.12)$$

AdaBoost (classification)

Model generation

Assign equal weight to each training instance $w^{(i)}=1/m$

For $j=1$ to N iterations:

- Apply learning algorithm to the weighted dataset

- Store resulting model

- Compute weighted error rate $r_j \longrightarrow r_j = \frac{\sum_{i=1}^m w^{(i)} \mathbb{1}_{\hat{y}_j^{(i)} \neq y^{(i)}}}{\sum_{i=1}^m w^{(i)}}$

- Compute the predictor's weight $\alpha_j = \eta \log \frac{1 - r_j}{r_j}$

- Update the instance weights: $w^{(i)} \leftarrow \begin{cases} w^{(i)} & \text{if } \hat{y}_j^{(i)} = y^{(i)} \\ w^{(i)} \exp(\alpha_j) & \text{if } \hat{y}_j^{(i)} \neq y^{(i)} \end{cases}$

- Normalize weight of all instances

AdaBoost

Classification

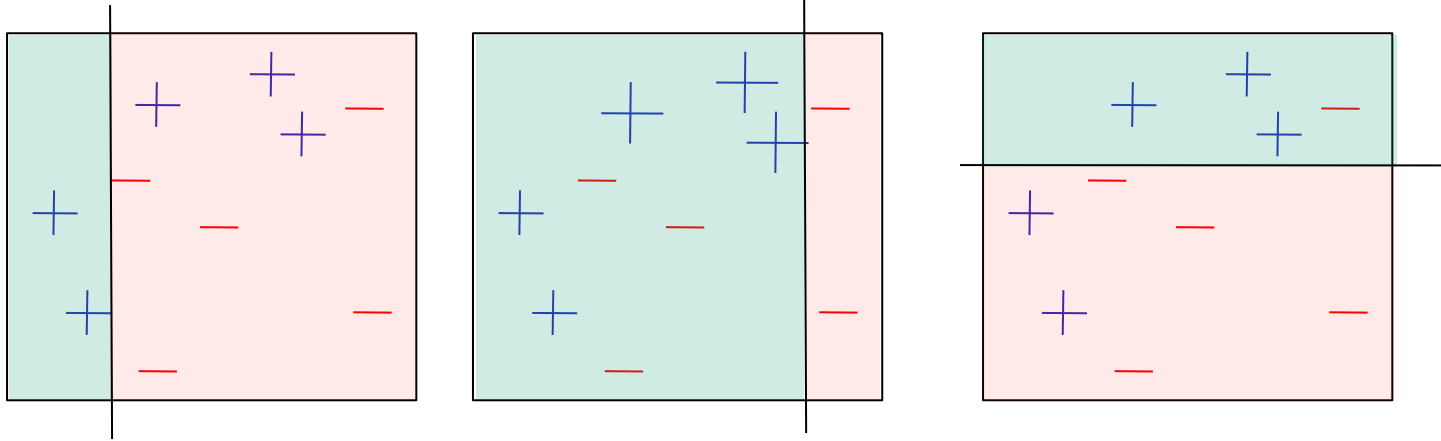
Input: data $\mathbf{x}$ to classify

Output: predicted class for input $\mathbf{x}$

- Compute the predictions of $\mathbf{x}$ for all the N predictors
- Weigh them using the predictor weights α_j
- The predicted class is the one that receives the majority of weighted votes:

$$\hat{y}(\mathbf{x}) = \underset{k}{\operatorname{argmax}} \sum_{j=1}^N \alpha_j \mathbf{1}_{\hat{y}_j(\mathbf{x}) = k}$$

Toy Example

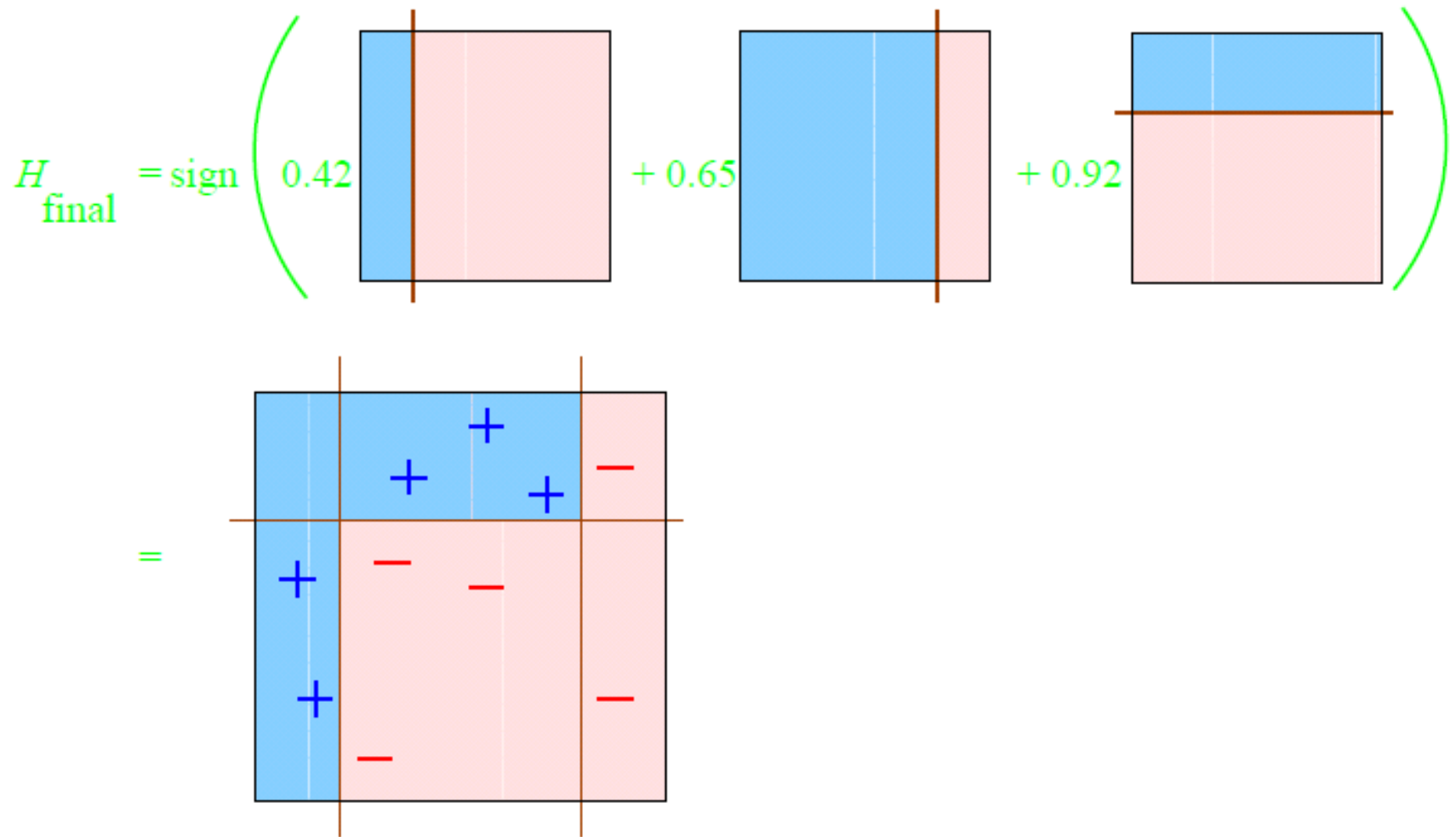


Final Classifier

$$H_{\text{final}} = \text{sign} \left(0.42 \begin{array}{|c|c|} \hline \text{blue} & \text{red} \\ \hline \end{array} + 0.65 \begin{array}{|c|c|} \hline \text{blue} & \text{red} \\ \hline \end{array} + 0.92 \begin{array}{|c|c|} \hline \text{blue} & \text{red} \\ \hline \end{array} \right)$$

The diagram illustrates the final classifier H_{final} as a weighted sum of three weak classifiers. Each weak classifier is represented by a square with a vertical decision boundary. The first classifier has a weight of 0.42 and a vertical boundary at the left. The second classifier has a weight of 0.65 and a vertical boundary at the right. The third classifier has a weight of 0.92 and a horizontal boundary at the top. The regions are colored blue or red, and the entire expression is enclosed in a large green parentheses.

Final Classifier



More on boosting

- Boosting needs weights ... but
- Can adapt learning algorithm ... or
- Can apply boosting without weights
 - ▣ resample with probability determined by weights
 - ▣ disadvantage: not all instances are used
 - ▣ advantage: if error > 0.5 , can resample again
- Stems from computational learning theory
- Theoretical result:
 - ▣ training error decreases exponentially
- Also:
 - ▣ works if base classifiers are not too complex, and
 - ▣ their error doesn't become too large too quickly

Gradient Boosting

Gradient boosting builds an ensemble of trees **one-by-one**, then the predictions of the individual trees **are summed**:

$$F(x) = F_n(x) = h_1(x) + \cdots + h_n(x)$$

The next decision tree tries to cover the discrepancy between the target function $y = f(x)$ and the current ensemble prediction **by reconstructing the residual**:

$$r_n(x) = y - F(x) = f(x) - F_n(x)$$

These are the parts that existing model F cannot do well.

The role of h_{n+1} is to compensate the shortcoming of existing model F_n . If the new model $F = F_n + h_{n+1}$ is still not satisfactory, we can add another regression tree...

How is this related to gradient descent?

Gradient Boosting

Define a loss function: $L(y_i, F(x_i)) = \frac{1}{2} (y_i - F(x_i))^2$

We want to minimize: $J = \sum_{i=1}^n L(y_i, F(x_i))$

By extending F_n ; note that $F(x_i)$ are just some numbers.

We can treat $F(x_i)$ as parameters and take derivatives:

$$\frac{\delta J}{\delta F(x_i)} = \frac{\delta \sum_{i=1}^n L(y_i, F(x_i))}{\delta F(x_i)} = \frac{\delta \sum_{i=1}^n (y_i - F(x_i))^2}{2 \delta F(x_i)} = -(y_i - F(x_i))$$

So we can interpret residuals as negative gradients and

$$\begin{aligned} F_{n+1}(x_i) &= F_n(x_i) + h_{n+1}(x_i) = F_n(x_i) + (y_i - F_n(x_i)) \\ &= F_n(x_i) - 1 \frac{\delta J}{\delta F(x_i)} = F_n(x_i) - \rho \frac{\delta J}{\delta F(x_i)} \end{aligned}$$

Gradient Boosting

For regression with **square loss** J :

residual $\Leftrightarrow$ negative gradient

fit h_k to residual $\Leftrightarrow$ fit h_k to negative gradient of J

update F based on residual $\Leftrightarrow$ update F based on
negative gradient of J

So we are actually updating our model using gradient descent! It turns out that the concept of gradients is more general and useful than the concept of residuals. So from now on, let's stick with gradients.

See this [link](#).

Loss Functions

The loss function is a function that maps an event or values of one or more variables onto a real number intuitively representing some "cost" associated with the event.

Regression Loss functions:

- L1 loss or Mean Absolute Errors (MAE)
- L2 Loss or Mean Square Error(MSE)
- Quadratic Loss

Classification Loss Functions:

- Log Loss (binary classification)
- Adaboost loss function (binary classification)
- Cross Entropy Loss

Boosting

In boosting, the trees are **built sequentially** such that each subsequent tree aims to **reduce the errors of the previous tree**. Each tree learns from its predecessors and updates the residual errors. Hence, the tree that grows next in the sequence will learn from an updated version of the residuals.

The base learners in boosting are **weak learners** in which the **bias is high**, and the predictive power is just a tad better than random guessing. Each of these weak learners contributes some vital information for prediction, enabling the boosting technique to produce a strong learner by effectively combining these weak learners. The final strong learner brings down both the bias and the variance.

Boosting

In contrast to bagging techniques like Random Forest, in which trees are grown to their maximum extent, boosting makes use of trees with fewer splits.

Such small trees, which are **not very deep**, are highly interpretable. Parameters like the number of trees or iterations, the rate at which the gradient boosting learns, and the depth of the tree, could be optimally selected through validation techniques like k-fold cross validation.

Having a large number of trees might lead to overfitting. So, it is necessary to carefully choose the stopping criteria for boosting.

Bagging vs Boosting

No clear winner; usually depends on the data

Bagging is computationally more efficient than boosting (note that bagging can train the M models in **parallel**, boosting can't)

Both **reduce variance** (and overfitting) by combining different models

The resulting model has **higher stability** as compared to the individual ones

Bagging usually can't reduce the bias, boosting can (note that in boosting, the training error steadily decreases)

Bagging usually performs better than boosting **if we don't have a high bias** and only want to reduce variance (i.e., if we are overfitting)

XGBoost

Extreme Gradient Boosting is based on Gradient Boosting: a special case of boosting where errors are minimized by gradient descent algorithm.

XGBoost and Gradient Boosting Machines (GBMs) are both ensemble tree methods that apply the principle of boosting weak learners using the gradient descent architecture.

XGBoost improves upon the base GBM framework through systems optimization and algorithmic enhancements.

What makes XGBoost so popular?

Speed and performance : Originally written in C++, it is comparatively faster than other ensemble classifiers.

Core algorithm is parallelizable : Because the core XGBoost algorithm is parallelizable it can harness the power of multi-core computers. It is also parallelizable onto GPU's and across networks of computers making it feasible to train on very large datasets as well.

Consistently outperforms other algorithm methods : It has shown better performance on a variety of machine learning benchmark datasets also with limited amount of input data.

Wide variety of tuning parameters : XGBoost internally has parameters for cross-validation, regularization, user-defined objective functions, missing values, tree parameters, scikit-learn & R & Julia & Java & JVM languages like Scala & platforms like Hadoop compatible API, etc.

XGBoost

System Optimization:

- Parallelization
- Tree Pruning
- Hardware Optimization

Algorithmic Enhancements:

- Regularization
- Sparsity Awareness
- Weighted Quantile Sketch
- Cross-validation

Blending/Stacking

Blending/Stacking

- To combine predictions of base learners, don't vote, use *meta learner*
 - ❑ Base learners: *level-0 models*
 - ❑ Meta learner: *level-1 model*
 - ❑ Predictions of base learners are input to meta learner
- Base learners are usually different schemes
- Can't use predictions on training data to generate data for level-1 model!
 - ❑ use hold-out set or cross-validation-like scheme
- Hard to analyze theoretically: “black magic”

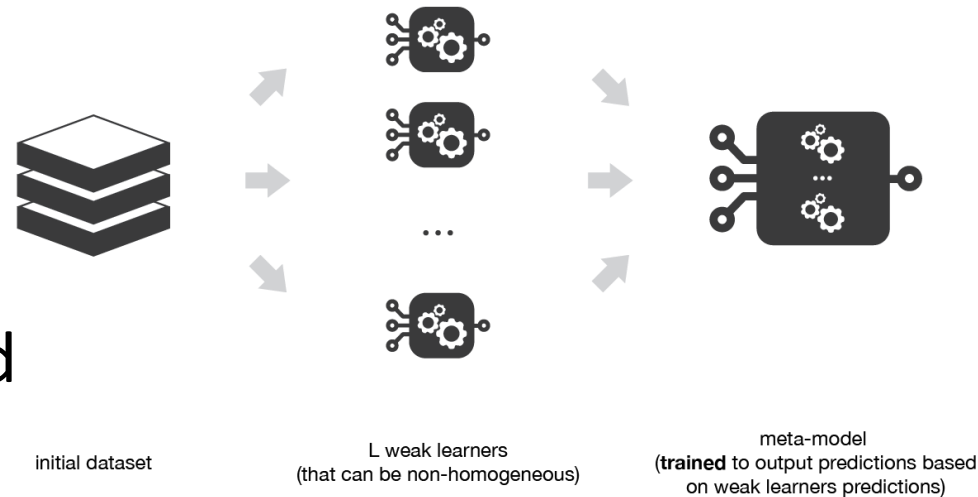
Blending/Stacking

If base learners can output probabilities, use those as input to meta learner instead

Which algorithm to use for meta learner?

- ❑ In principle, any learning scheme
- ❑ Prefer “relatively global, smooth” model
 - ✓ Base learners do most of the work
 - ✓ Reduces risk of overfitting

Stacking can be applied to numeric prediction too



Blending

Assuming that we want to fit a blending ensemble composed of L weak learners. Then we have to follow the steps thereafter:

- split the training data in two folds
- choose L weak learners and fit them to data of the first fold
- for each of the L weak learners, make predictions for observations in the second fold
- fit the meta-model on the second fold, using predictions made by the weak learners as inputs

Stacking

Assuming that we want to fit a stacking ensemble composed of L weak learners:

- Train the L **Base-Models**: the k -fold cross-validation procedure is performed on each base model of interest, and all of the out-of-fold predictions are collected
- Train the **Meta-Model**: it is trained on the predictions made by the other models. This meta-model may or may not also take the input data for each example as input when making predictions.

The job of this model is to learn how to best combine and correct the predictions made by the other models using their out-of-fold predictions.

StackingClassifier

```
class sklearn.ensemble.StackingClassifier(estimators,  
final_estimator=None, *, cv=None, stack_method='auto',  
n_jobs=None, passthrough=False, verbose=0)
```

- ***estimators*** *list of (str, estimator)*
Base estimators which will be stacked together. Each element of the list is defined as a tuple of string (i.e. name) and an estimator instance. An estimator can be set to 'drop' using set_params.
- ***final_estimator*** *estimator, default=None*
A classifier which will be used to combine the base estimators. The default classifier is a LogisticRegression.
- ***cv*** *int, cross-validation generator or an iterable, default=None*
Determines the cross-validation splitting strategy used in cross_val_predict to train final_estimator.
- ***passthrough*** *bool, default=False*
When False, only the predictions of estimators will be used as training data for final_estimator. When True, the final_estimator is trained on the predictions as well as the original training data.